

Name: **R Priya Dharshini**

Roll: **CH.SC.U4CYS25026**

ACTIVITY-2

Activity 2: Part 2

1. Write C program for the following:

- Take four short or long sentences. Form a circular linked list using strings in the sentences sequentially.
- Circular shift the statements (Analyze and implement all possible circular combinations)
- Pick any shifted version. Prepare 2D matrix with elements as letter count difference computed from pairwise tokens in the sentences. Compute the correlation matrix using it.

2. Write C program for the following:

- I want to pick an element out of a stack without disturbing the order. Hint: Use an extra stack
- Need to check whether parenthesis, curly braces and brackets are balanced or not in my program using stack. Hint: Conditionally use 'push' and 'pop'
- Reverse the list using stack-queue combination (Single stack and queue only allowed)
Example: List = [1, 2, 7, 3], Reverse = [3, 7, 2, 1]
- Mirror the list using stack-queue combination (Single stack and queue only allowed)
Example: List = [1, 2, 7, 3], Reverse = [1, 2, 7, 3, 3, 7, 2, 1]
- Multiple copying list elements using a single queue and its relevant operations.
Example: List = [1, 2, 7, 3], 3-copy List = [1, 1, 1, 2, 2, 2, 7, 7, 7, 3, 3, 3]

ANSWER 1(a):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
/*
```

PROGRAM:

Create a Circular Linked List using four sentences.

OBJECTIVE:

- Accept four sentences from the user.
- Store each sentence in a node.
- Connect the last node to the first node.
- Display the circular linked list.

DATA STRUCTURE:

Circular Singly Linked List

TIME COMPLEXITY:

Insertion : $O(n)$

Traversal : $O(n)$

SPACE COMPLEXITY:

$O(n)$

```

*/

/* Structure representing a node in the circular linked list */
struct Node
{
    char sentence[200];    // Stores one complete sentence
    struct Node *next;    // Stores address of next node
};

/* Function to create a new node */
struct Node* createNode(char str[])
{
    /* Allocate memory dynamically */
    struct Node *newNode =
        (struct Node*)malloc(sizeof(struct Node));

    /* Copy sentence into node */
    strcpy(newNode->sentence, str);

    /* Initially next pointer is NULL */
    newNode->next = NULL;

    return newNode;
}

/* Function to display circular linked list */
void display(struct Node *head)
{
    /* If list is empty */
    if(head == NULL)
    {
        printf("List is Empty\n");
        return;
    }

    struct Node *temp = head;

    printf("\nCircular Linked List Contents:\n");

    /*
        Traverse until we reach
        the starting node again
    */
    do
    {
        printf("%s\n", temp->sentence);

        temp = temp->next;

    } while(temp != head);
}

int main()
{
    struct Node *head = NULL;

```

```

struct Node *tail = NULL;

char input[4][200];

printf("Enter 4 Sentences:\n");

/* Read four sentences */
for(int i=0;i<4;i++)
{
    fgets(input[i],200,stdin);

    /* Remove newline character */
    input[i][strcspn(input[i],"\\n")] = '\\0';

    /* Create node */
    struct Node *newNode =
        createNode(input[i]);

    /*
        If first node,
        initialize head and tail
    */
    if(head == NULL)
    {
        head = tail = newNode;
    }
    else
    {
        /*
            Attach new node
            after current tail
        */
        tail->next = newNode;

        /* Update tail */
        tail = newNode;
    }
}

/*
    Connect last node to first node
    to make list circular
*/
tail->next = head;

display(head);

printf("\\nProgram Executed Successfully\\n");

return 0;
}

```

OUTPUT

Output

Enter 4 Sentences:

first sentence

second sentence

third sentence

fourth sentence

Circular Linked List Contents:

first sentence

second sentence

third sentence

fourth sentence

Program Executed Successfully

=== Code Execution Successful ===

ANSWER 1(b):

```
#include <stdio.h>
#include <string.h>
```

```
/*
```

```
=====
PROGRAM:
```

Generate all possible circular shifts of 4 sentences.

OBJECTIVE:

1. Accept four sentences.
2. Rotate sentences circularly.
3. Display every possible circular arrangement.

EXAMPLE:

Original:

S1

S2

S3

S4

Shift 1:

S2

S3

S4

S1

Shift 2:

S3

S4
S1
S2

Shift 3:

S4
S1
S2
S3

TIME COMPLEXITY:

$O(n^2)$

SPACE COMPLEXITY:

$O(n)$

```
=====
*/

int main()
{
    char sentence[4][200];

    printf("Enter 4 Sentences:\n");

    /* Read all four sentences */
    for(int i=0;i<4;i++)
    {
        fgets(sentence[i],200,stdin);

        /* Remove newline character */
        sentence[i][strcspn(sentence[i],"\n")] = '\0';
    }

    printf("\nAll Possible Circular Shifts:\n");

    /*
        Outer loop controls
        the amount of rotation
    */
    for(int shift=0; shift<4; shift++)
    {
        printf("\nShift %d:\n",shift);

        /*
            Print sentences in
            circular shifted order
        */
        for(int i=0;i<4;i++)
        {
            /*
                Modulus operator (%)
                automatically wraps
                index back to beginning
            */
            printf("%s\n",
```

```

        sentence[(i+shift)%4]);
    }
}

printf("\nProgram Executed Successfully\n");

return 0;
}

```

OUTPUT:

Output	
Enter 4 Sentences:	Shift 2:
amrita	vidhyapeetham
vishwa	chennai
vidhyapeetham	amrita
chennai	vishwa
All Possible Circular Shifts:	
Shift 0:	Shift 3:
amrita	chennai
vishwa	amrita
vidhyapeetham	vishwa
chennai	vidhyapeetham
Shift 1:	
vishwa	
vidhyapeetham	
chennai	
amrita	Program Executed Successfully

ANSWER 1(c):

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

```

```

/*
=====

```

PROGRAM:
Difference Matrix and Correlation Matrix

OBJECTIVE:

1. Read 4 sentences.
2. Calculate length of each sentence.
3. Create Difference Matrix using $|\text{Length}(i) - \text{Length}(j)|$.
4. Create Correlation Matrix using

1/(1 + Difference).

5. Display both matrices.

TIME COMPLEXITY : $O(n^2)$

SPACE COMPLEXITY: $O(n^2)$

```
*/  
  
int main()  
{  
    char sentence[4][200];  
  
    int length[4];  
  
    int difference[4][4];  
  
    float correlation[4][4];  
  
    printf("Enter 4 Sentences:\n");  
  
    /* Read 4 sentences */  
    for(int i = 0; i < 4; i++)  
    {  
        fgets(sentence[i], sizeof(sentence[i]), stdin);  
  
        /* Remove newline character */  
        sentence[i][strcspn(sentence[i], "\n")] = '\0';  
    }  
  
    /* Find length of each sentence */  
    for(int i = 0; i < 4; i++)  
    {  
        length[i] = strlen(sentence[i]);  
    }  
  
    /* Build Difference Matrix */  
    for(int i = 0; i < 4; i++)  
    {  
        for(int j = 0; j < 4; j++)  
        {  
            difference[i][j] =  
                abs(length[i] - length[j]);  
        }  
    }  
  
    /* Build Correlation Matrix */  
    for(int i = 0; i < 4; i++)  
    {  
        for(int j = 0; j < 4; j++)  
        {  
            correlation[i][j] =  
                1.0f / (1 + difference[i][j]);  
        }  
    }  
}
```

```
printf("\nLength of Sentences:\n");

for(int i = 0; i < 4; i++)
{
    printf("Sentence %d Length = %d\n",
        i + 1, length[i]);
}

printf("\nDifference Matrix:\n");

for(int i = 0; i < 4; i++)
{
    for(int j = 0; j < 4; j++)
    {
        printf("%4d ", difference[i][j]);
    }
    printf("\n");
}

printf("\nCorrelation Matrix:\n");

for(int i = 0; i < 4; i++)
{
    for(int j = 0; j < 4; j++)
    {
        printf("%.2f ", correlation[i][j]);
    }
    printf("\n");
}

printf("\nProgram Executed Successfully\n");

return 0;
}
```

OUTPUT:

Output

Enter 4 Sentences:

SUBJECT: DSA

DATA STRUCTURE ALGORITHM

CYBER SECURITY STUDENT

AMRITA VISHWA VIDHYAPEETHAM, CHENNAI CAMPUS

Length of Sentences:

Sentence 1 Length = 12

Sentence 2 Length = 24

Sentence 3 Length = 22

Sentence 4 Length = 43

Difference Matrix:

0	12	10	31
12	0	2	19
10	2	0	21
31	19	21	0

Correlation Matrix:

1.00	0.08	0.09	0.03
0.08	1.00	0.33	0.05
0.09	0.33	1.00	0.05
0.03	0.05	0.05	1.00

Program Executed Successfully

=== Code Execution Successful ===

ANSWER 2(a):

```
#include <stdio.h>
```

```
#define MAX 100
```

```
/*
```

```
=====
```

PROGRAM:

Access an element from stack without disturbing order.

OBJECTIVE:

1. Store elements in stack.
2. Access any position from top.
3. Restore original stack order.

CONCEPT:

Use an auxiliary stack.

TIME COMPLEXITY:

$O(n)$

SPACE COMPLEXITY:

$O(n)$

*/

```
int stack[MAX];
int top=-1;
```

```
/* Push operation */
void push(int value)
{
    stack[++top]=value;
}
```

```
/* Pop operation */
int pop()
{
    return stack[top--];
}
```

```
int main()
{
    int n;

    printf("Enter Number of Elements: ");
    scanf("%d",&n);
```

```
    printf("Enter Stack Elements:\n");
```

```
    /* Insert elements */
    for(int i=0;i<n;i++)
    {
        int x;

        scanf("%d",&x);

        push(x);
    }
```

```
    int position;
```

```
    printf("Position from Top: ");
    scanf("%d",&position);
```

```
    int temp[MAX];
```

```
    int tempTop=-1;
```

```
    /*
        Move elements above
        required position
```

```

        into temporary stack
    */
    for(int i=1;i<position;i++)
    {
        temp[++tempTop]=pop();
    }

    /*
    Required element
    now appears at top
    */
    int required = stack[top];

    /*
    Restore original order
    */
    while(tempTop!=-1)
    {
        push(temp[tempTop--]);
    }

    printf("Required Element = %d\n",
        required);

    printf("\nProgram Executed Successfully\n");

    return 0;
}

```

OUTPUT:

Output

Enter Number of Elements: 5

Enter Stack Elements:

3

7

2

4

8

Position from Top: 2

Required Element = 4

Program Executed Successfully

=== Code Execution Successful ===

ANSWER 2(b):

```
#include <stdio.h>
#include <string.h>
```

```
/*
```

```
=====
```

PROGRAM:

Balanced Parentheses Checker using Stack

OBJECTIVE:

1. Accept a complete expression/sentence.
2. Check whether (), {} and [] are balanced.
3. Display Balanced or Not Balanced.

DATA STRUCTURE:

Stack

TIME COMPLEXITY:

O(n)

SPACE COMPLEXITY:

O(n)

```
=====
```

```
*/
```

```
/* Stack declaration */
char stack[100];
```

```
int top = -1;
```

```
/* Push operation */
void push(char ch)
{
    stack[++top] = ch;
}
```

```
/* Pop operation */
char pop()
{
    return stack[top--];
}
```

```
/* Function to check matching brackets */
int match(char open, char close)
{
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
}
```

```
int main()
{
    char exp[200];

    printf("Enter Expression: ");
```

```

/* Read complete sentence including spaces */
fgets(exp, sizeof(exp), stdin);

/* Remove newline character */
exp[strcspn(exp, "\n")] = '\0';

/* Scan entire expression */
for(int i = 0; exp[i] != '\0'; i++)
{
    /* Push opening brackets */
    if(exp[i] == '(' ||
       exp[i] == '{' ||
       exp[i] == '[')
    {
        push(exp[i]);
    }

    /* Process closing brackets */
    else if(exp[i] == ')' ||
            exp[i] == '}' ||
            exp[i] == ']')
    {
        /* No matching opening bracket */
        if(top == -1)
        {
            printf("\nNot Balanced Expression\n");
            return 0;
        }

        /* Check matching pair */
        if(!match(pop(), exp[i]))
        {
            printf("\nNot Balanced Expression\n");
            return 0;
        }
    }
}

/* If stack is empty, all brackets matched */
if(top == -1)
{
    printf("\nBalanced Expression\n");
}
else
{
    printf("\nNot Balanced Expression\n");
}

printf("\nProgram Executed Successfully\n");

return 0;
}

```

OUTPUT:

Output

```
Enter Expression: im a (cyber security) student at {amrita
```

```
Not Balanced Expression
```

```
Program Executed Successfully
```

```
=== Code Execution Successful ===
```

Output

```
Enter Expression: im a (cyber security) student at {amrita}
```

```
Balanced Expression
```

```
Program Executed Successfully
```

```
=== Code Execution Successful ===
```

ANSWER 2(c):

```
#include <stdio.h>
```

```
/*
```

```
=====
```

PROGRAM:

Reverse a List using One Stack and One Queue

OBJECTIVE:

1. Read list elements.
2. Store elements in queue.
3. Transfer elements to stack.
4. Print stack elements.
5. Obtain reverse order.

TIME COMPLEXITY:

$O(n)$

SPACE COMPLEXITY:

$O(n)$

```
=====
```

```
*/
```

```
int main()
```

```
{
```

```

int n;

printf("Enter Number of Elements: ");
scanf("%d",&n);

/* Queue representation */
int queue[100];

/* Stack representation */
int stack[100];

int top = -1;

printf("Enter Elements:\n");

/*
    Read elements into queue
    maintaining original order
*/
for(int i=0;i<n;i++)
{
    scanf("%d",&queue[i]);
}

/*
    Transfer elements from queue
    into stack

    Stack automatically reverses
    order because of LIFO property
*/
for(int i=0;i<n;i++)
{
    stack[++top] = queue[i];
}

printf("\nReversed List:\n");

/*
    Pop elements from stack
    to obtain reverse order
*/
while(top!=-1)
{
    printf("%d ",stack[top--]);
}

printf("\n\nProgram Executed Successfully\n");

return 0;
}

```

OUTPUT:

Output

Enter Number of Elements: 4

Enter Elements:

5

3

2

7

Reversed List:

7 2 3 5

Program Executed Successfully

=== Code Execution Successful ===

ANSWER 2(d):

```
#include <stdio.h>
```

```
/*
```

```
=====
```

PROGRAM:

Generate Mirror List

OBJECTIVE:

Create a mirror image of a list.

EXAMPLE:

Input:

1 2 7 3

Output:

1 2 7 3 3 7 2 1

TIME COMPLEXITY:

$O(n)$

SPACE COMPLEXITY:

$O(n)$

```
=====
```

```
*/
```

```
int main()
```

```
{
```

```
    int n;
```

```

printf("Enter Number of Elements: ");
scanf("%d",&n);

int queue[100];
int stack[100];

int top=-1;

printf("Enter Elements:\n");

/*
    Store elements in queue
    and simultaneously push
    into stack
*/
for(int i=0;i<n;i++)
{
    scanf("%d",&queue[i]);

    stack[++top]=queue[i];
}

printf("\nMirror List:\n");

/*
    Print original sequence
*/
for(int i=0;i<n;i++)
{
    printf("%d ",queue[i]);
}

/*
    Print reverse sequence
    obtained from stack
*/
while(top!=-1)
{
    printf("%d ",stack[top--]);
}

printf("\n\nProgram Executed Successfully\n");

return 0;
}

```

OUTPUT:

Output

Enter Number of Elements: 5

Enter Elements:

9

3

4

6

5

Mirror List:

9 3 4 6 5 5 6 4 3 9

Program Executed Successfully

=== Code Execution Successful ===

ANSWER 2(e):

```
#include <stdio.h>
```

```
/*
```

```
=====
```

PROGRAM:

Multiple Copy List

OBJECTIVE:

Create three copies of every
element in the given list.

EXAMPLE:

Input:

1 2 7 3

Output:

1 1 1 2 2 2 7 7 7 3 3 3

TIME COMPLEXITY:

$O(n \times k)$

SPACE COMPLEXITY:

$O(1)$

where k = number of copies

```
=====
```

```
*/
```

```
int main()
```

```
{
    int n;

    printf("Enter Number of Elements: ");
    scanf("%d",&n);

    int arr[100];

    printf("Enter Elements:\n");

    /*
       Read all elements
       into array
    */
    for(int i=0;i<n;i++)
    {
        scanf("%d",&arr[i]);
    }

    printf("\nMultiple Copy List:\n");

    /*
       Traverse every element
    */
    for(int i=0;i<n;i++)
    {
        /*
           Print each element
           three times
        */
        for(int j=0;j<3;j++)
        {
            printf("%d ",arr[i]);
        }
    }

    printf("\n\nProgram Executed Successfully\n");

    return 0;
}
```

OUTPUT:

Output

Enter Number of Elements: 3

Enter Elements:

5 6 7

Multiple Copy List:

5 5 5 6 6 6 7 7 7

Program Executed Successfully

=== Code Execution Successful ===